

Unit – 3

Class and Objects

Object Oriented Programming (CSC166)

BSc CSIT 2nd Semester
Amrit Campus

Prepared By:
Dabbal Singh Mahara

CLASS AND OBJECTS

- A class is a blueprint(model) for the object. Object is an instance (variable)of a class
- We can think of a class as a sketch (prototype) of a house. It contains all the details about the floors, doors, windows, etc. Based on these descriptions we build the house. House is the object.
- Once a class has been defined , we can create any number of objects of the same class.
- So, we can say that class is a user defined data type.

CREATING A CLASS IN C++

- A class in C++ is the building block of Object-Oriented programming.
- A class is defined in C++ using keyword class followed by the name of the class.
- The body of the class is defined inside the curly brackets and terminated by a semicolon at the end.
- The body of a class encapsulates data and functions called the class members.
- The data components of the class are called data members and the function components are called member functions.

Creating a class in C++

```
keyword  
class classname ← classname  
{  
    Access specifiers: ← //private/public/protected  
    Data members/variables;    //variables  
    Member functions () {}    //methods  
}; ← //end of class with a semicolon
```

- In C++, data can be hidden by making it **private**.
- The members that are declared private can be accessed only from within the class.
- When access specifiers are not specified, members will be private by default.
- **public** members can be accessed from outside the class also.
- **protected** members can be accessed from the child class also.
- Usually within a class data are private and functions are public

Creating a class in C++

```
class Room {  
    public:  
        double length;  
        double breadth;  
        double height;  
  
        double calculateArea(){  
            return length * breadth;  
        }  
  
        double calculateVolume(){  
            return length * breadth * height;  
        }  
  
};
```

Here, we defined a class named `Room`.

The variables `length`, `breadth`, and `height` declared inside the class are known as **data members**. And, the functions `calculateArea()` and `calculateVolume()` are known as **member functions** of a class.

Example: Declaring a class rectangle

```
class rectangle
{
    private:
        int length, breadth;
    public:
        void setdata(int l, int b)
        {
            length = l;
            breadth= b;
        }
        void showdata()
        {
            cout<<"length="<<length<<endl<<"breadth="<<breadth<<endl;
        }
        int findArea()
        {
            return length*breadth;
        }
        int findPerimeter()
        {
            return 2*length*breadth;
        }
};
```

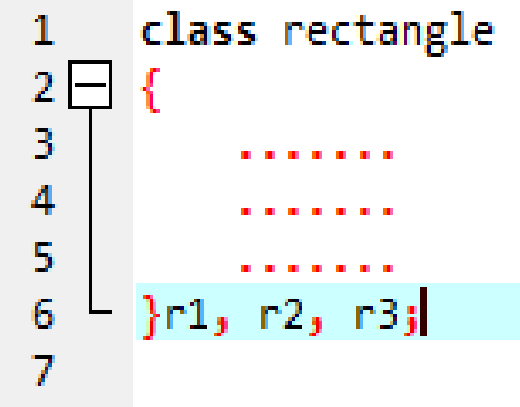
CREATING OBJECTS

- When a class is defined, only the specification for the object is defined; no memory or storage is allocated.
- When a class is defined, only the specification for the object is defined; no memory or storage is allocated.
- **Syntax:** `class_name object_name; // instantiating object`
 - **E.g.:** `rectangle r; // it creates a variable(object) r of type rectangle.`
- We can create any number of objects form the same class
 - **E.g.:** `rectangle r1, r2,....;`

Alternative way to create objects:

- Objects can also be created when a class is defined by placing their names immediately after the closing brace.
- E.g.,:

```
1 class rectangle
2 {
3     .....
4     .....
5     .....
6 }r1, r2, r3;
7
```



- **Note:** size of the object is the sum of the size of all the data members declared in the class.

- **Accessing class members:**

- When an objects of the class is created then the members are accessed using the ‘.’ dot operator(also called member access operator) except the private members.

- **Syntax:** object_name.data_member_name;

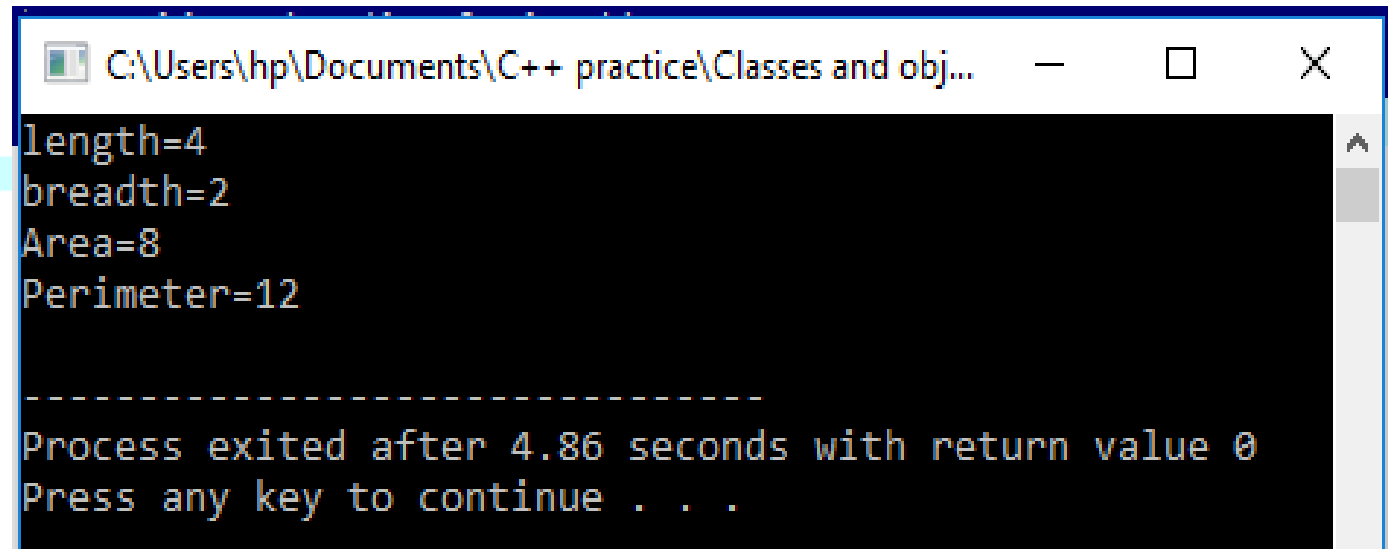
- object_name.function_member_name(argements....);

- **For example:** r.setData(4, 2);

Example: Program to find the area and perimeter of rectangle

```
1 // A complete program using class
2 #include<iostream>
3 using namespace std;
4 class rectangle
5 {
6     private:
7         int length, breadth;
8     public:
9         void setdata(int l, int b)
10        {
11            length = l;
12            breadth= b;
13        }
14        void showdata()
15        {
16            cout<<"length="<<length<<endl<<"breadth="<<breadth<<endl;
17        }
18        int findArea()
19        {
20            return length*breadth;
21        }
22        int findPerimeter()
23        {
24            return 2*(length+breadth);
25        }
26    };
27 int main()
28 {
29     rectangle r;
30     r.setdata(4,2);
31     r.showdata();
32     cout<<"Area="<<r.findArea()<<endl;
33     cout<<"Perimeter="<<r.findPerimeter()<<endl;
34     return 0;
35 }
```

• Output:



The screenshot shows a Windows command prompt window titled "C:\Users\hp\Documents\C++ practice\Classes and obj...". The output of the program is displayed as follows:

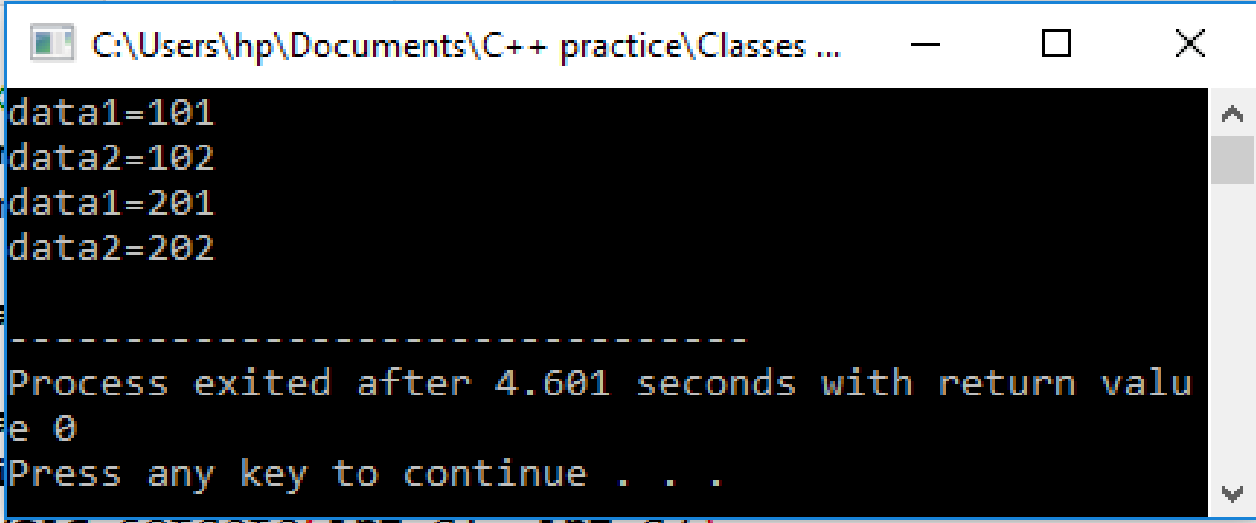
```
length=4
breadth=2
Area=8
Perimeter=12

-----
Process exited after 4.86 seconds with return value 0
Press any key to continue . . .
```

Example2: simple program to set and show the data by using the concept of class

```
1 //Simple program to illustrate class and object
2 #include<iostream>
3 using namespace std;
4 class simple
5 {
6     private:
7         int data1;
8     private: int data2;
9     public:
10         void setdata(int d1, int d2)
11         {
12             data1 = d1;
13             data2 = d2;
14         }
15         void showdata()
16         {
17             cout<<"data1="<<data1<<endl;
18             cout<<"data2="<<data2<<endl;
19         }
20 };
21 int main()
22 {
23     simple s1, s2; // simple act as a data type and s1 and s2 as a variable
24     s1.setdata(101, 102); // sets 101 to d1 and 102 to d2
25     s2.setdata(201, 202); //sets 201 to d1 and 2020 to d2
26     s1.showdata();
27     s2.showdata();
28 }
```

• Output:

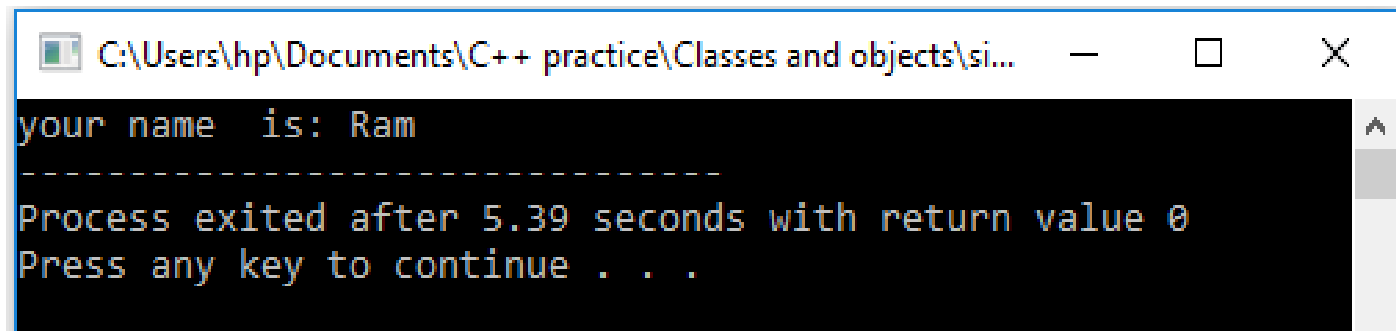


```
C:\Users\hp\Documents\C++ practice\Classes ...
data1=101
data2=102
data1=201
data2=202
-----
Process exited after 4.601 seconds with return value 0
Press any key to continue . . .
```

Example3: Program to demonstrate the class and object

```
1 // C++ program to demonstrate accessing of data members
2 #include <iostream>
3 using namespace std;
4 class name
5 {
6     // Access specifier
7     public:
8
9     // Data Members
10    string firstName;
11
12    // Member Functions()
13    void printname()
14    {
15        cout << "your name is: " << firstName;
16    }
17 };
18
19 int main() {
20
21     // Declare an object of class geeks
22     name obj1;
23
24     // accessing data member
25     obj1.firstName = "Ram";
26
27     // accessing member function
28     obj1.printname();
29     return 0;
30 }
```

• Output:



```
C:\Users\hp\Documents\C++ practice\Classes and objects\si...
your name is: Ram
-----
Process exited after 5.39 seconds with return value 0
Press any key to continue . . .
```

Practice Work 1:

Write a program designing a class to represent Item information.

Include the following members

- Data members

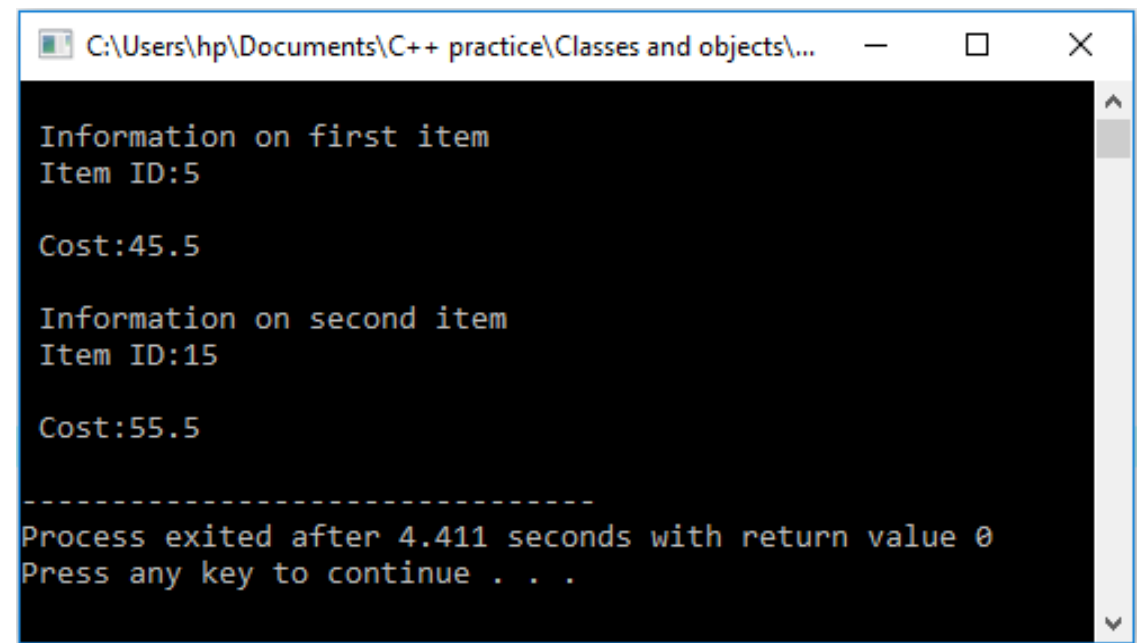
- Itemid
- cost

- Member functions

- Setdata
- showdata

- **Program:**

```
#include<iostream>
using namespace std;
class iteminfo
{
    private:
        int itemid;
        float cost;
    public:
        void setdata(int it, float cst)
        {
            itemid = it;
            cost = cst;
        }
        void showdata()
        {
            cout<<"\n Item ID:"<<itemid<<endl;
            cout<<"\n Cost:"<<cost<<endl;
        }
};
int main()
{
    iteminfo i1, i2;
    i1.setdata(5, 45.5);
    i2.setdata(15, 55.5);
    cout<<"\n Information on first item";
    i1.showdata();
    cout<<"\n Information on second item";
    i2.showdata();
    return 0;
}
```



The screenshot shows a Windows command prompt window titled "C:\Users\hp\Documents\C++ practice\Classes and objects\...". The output of the program is displayed as follows:

```
Information on first item
Item ID:5

Cost:45.5

Information on second item
Item ID:15

Cost:55.5

-----
Process exited after 4.411 seconds with return value 0
Press any key to continue . . .
```

DEFINING MEMBER FUNCTION

- The data member of a class is declared within the body of class.

However, the member functions can be defined in one of the two places:

- **Inside the class defining:** Definition and declaration is placed in the same place i.e., inside the class definition.
- **Outside the class definition:**
 - Declaration is placed inside the class definition and
 - But the definition is provided outside of the class by using scope resolution operator.

– Defining member function Outside the class definition

- Declaration is placed inside the class definition and
- But the definition is provided outside of the class by using scope resolution operator as:
- **Syntax:**

```
Return_type class_name::function_name(argument declaration)

{

    //Function body

}
```

Example:

```
// A complete program using class
#include<iostream>
using namespace std;
class rectangle
{
    private:
        int length, breadth;
    public:
        void setdata(int l, int b)
        {
            length = l;
            breadth= b;
        }
        void showdata();
        int findArea();
        int findPerimeter();
};

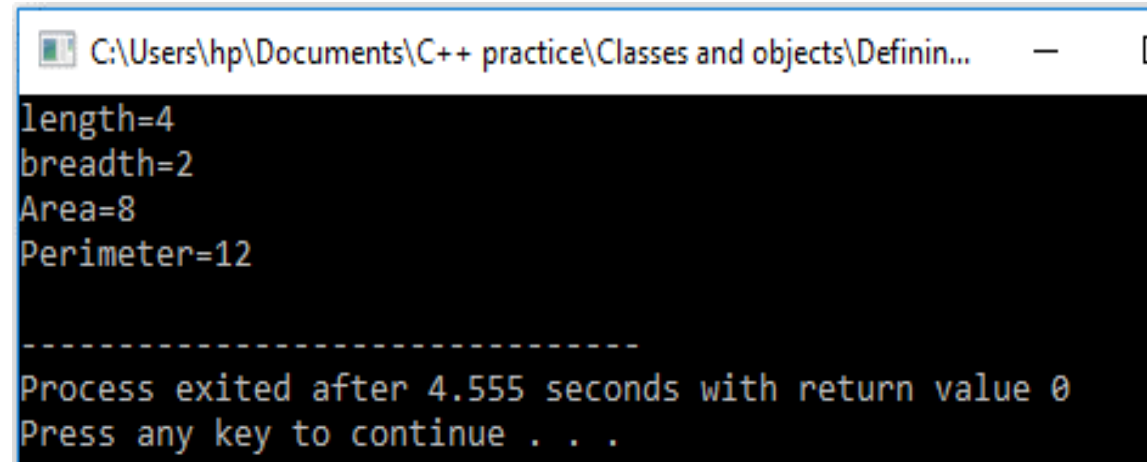
void rectangle:: showdata()
{
    cout<<"length="<<length<<endl<<"breadth="<<breadth<<endl;
}

int rectangle:: findArea()
{
    return length*breadth;
}

int rectangle:: findPerimeter()
{
    return 2*(length+breadth);
}

int main()
{
    rectangle r;
    r.setdata(4,2);
    r.showdata();
    cout<<"Area="<<r.findArea()<<endl;
    cout<<"Perimeter="<<r.findPerimeter()<<endl;
    return 0;
}
```

Output:



C:\Users\hp\Documents\C++ practice\Classes and objects\Definin...

```
length=4
breadth=2
Area=8
Perimeter=12

-----
Process exited after 4.555 seconds with return value 0
Press any key to continue . . .
```

MAKING OUTER FUNCTION INLINE

- In C++, all the member functions defined inside the class are treated as inline whereas the function defined outside the class are not inline.
- The member function defined outside the class can be made inline too. It is made inline by prefixing the keyword **inline** to the function definition as follows:

- **Syntax:**

```
class class_name
{
    private:
        //.....
    public:
        return_type member_fuction(args..)
        //.....

};
inline return_type class_name::member_function(args....)
{
    //Function body
}
```

- A member function can be called by using its name from another member function of the same class. This is known as nesting of member functions.

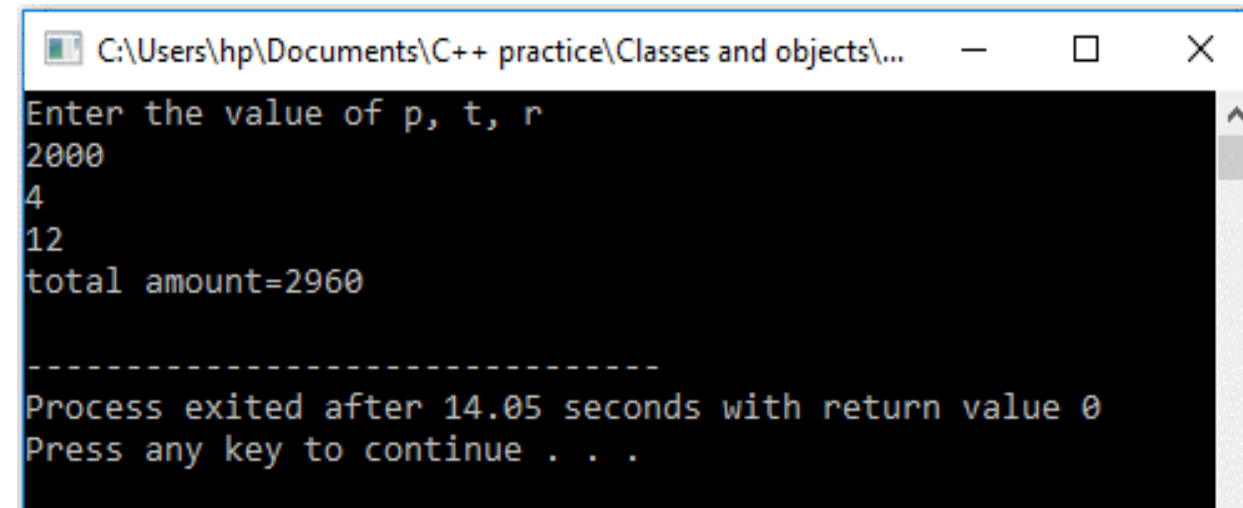
```
//nesting of member functions
class class_name
{
    private:
        // data members;
        return_type function1(args...)
        {
            .....
            .....
        }
    public:
        //data members;
        return_type function2(args..)
        {
            .....
            .....
            function1(args...); // nesting
        }
};

int main()
{
    class_name c;
    c.function2(args...);
    .....
    .....
}
```

Example:

```
//nesting of member function (findinterest insdie findtotal)
#include<iostream>
using namespace std;
class total
{
    private:
        float principle, time, rate;
        float findinterest()
        {
            return principle*time*rate/100;
        }
    public:
        void setdata (float p, float t, float r)
        {
            principle =p;
            time =t;
            rate =r;
        }
        float findtotal()
        {
            return principle +findinterest();
        }
};
int main()
{
    float p, t, r, a;
    total ta;
    cout<< "Enter the value of p, t, r"<<endl;
    cin>>p>>t>>r;
    ta.setdata(p,t,r);
    a = ta.findtotal();
    cout<<"total amount="<< a<< endl;
    return 0;
}
```

Output:

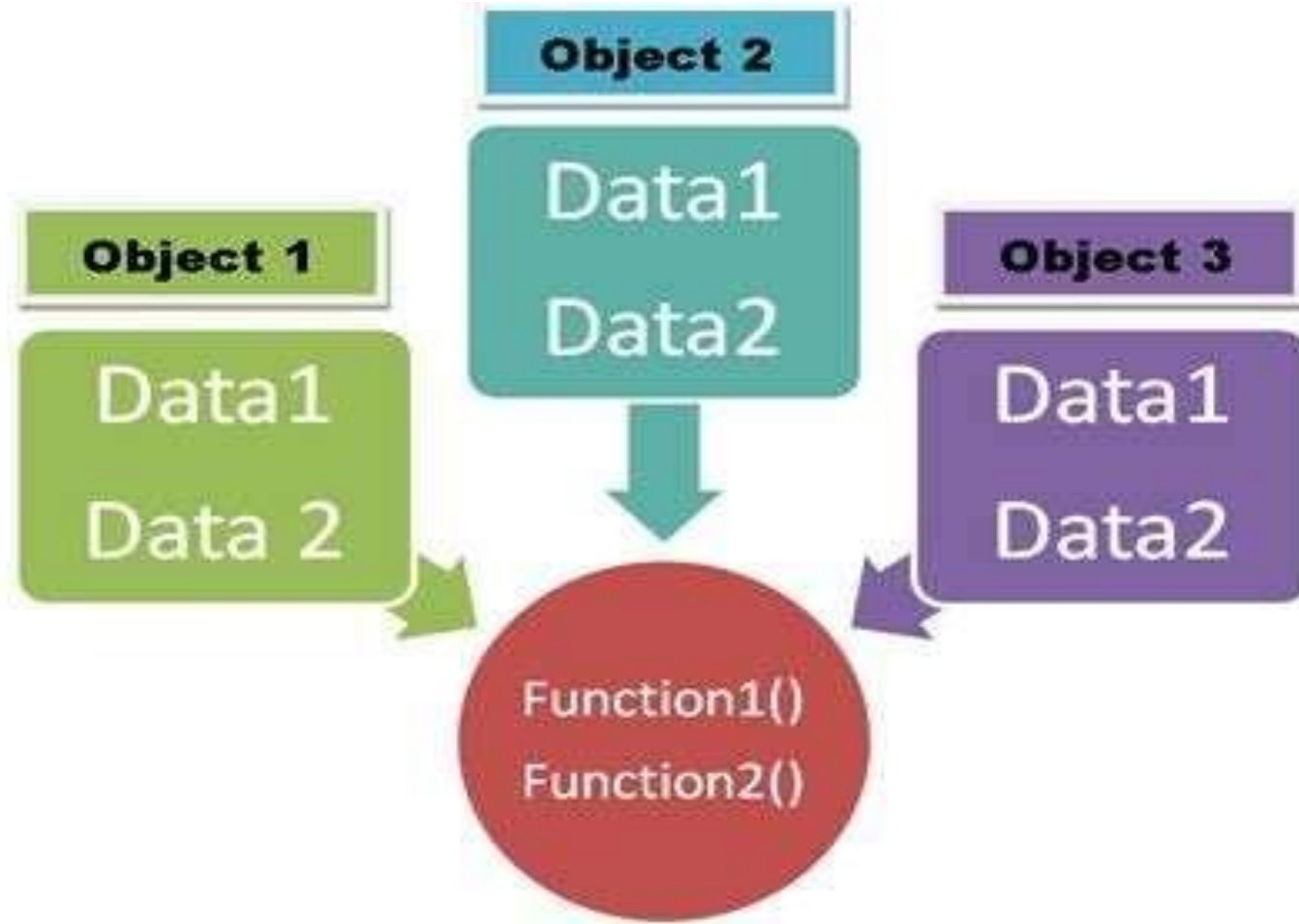


The screenshot shows a Windows command prompt window titled "C:\Users\hp\Documents\C++ practice\Classes and objects\...". The window contains the following text:

```
Enter the value of p, t, r
2000
4
12
total amount=2960
-----
Process exited after 14.05 seconds with return value 0
Press any key to continue . . .
```

MEMORY ALLOCATION FOR OBJECTS

- For each object, the memory space for data members is allocated separately because the data members will hold different data values for different objects.
- However, all the objects in a given class use the same member function.
- Hence, the member functions are created and placed in memory only once when they are defined as a part of class specification and no separate space is allocated for member functions when objects are created.



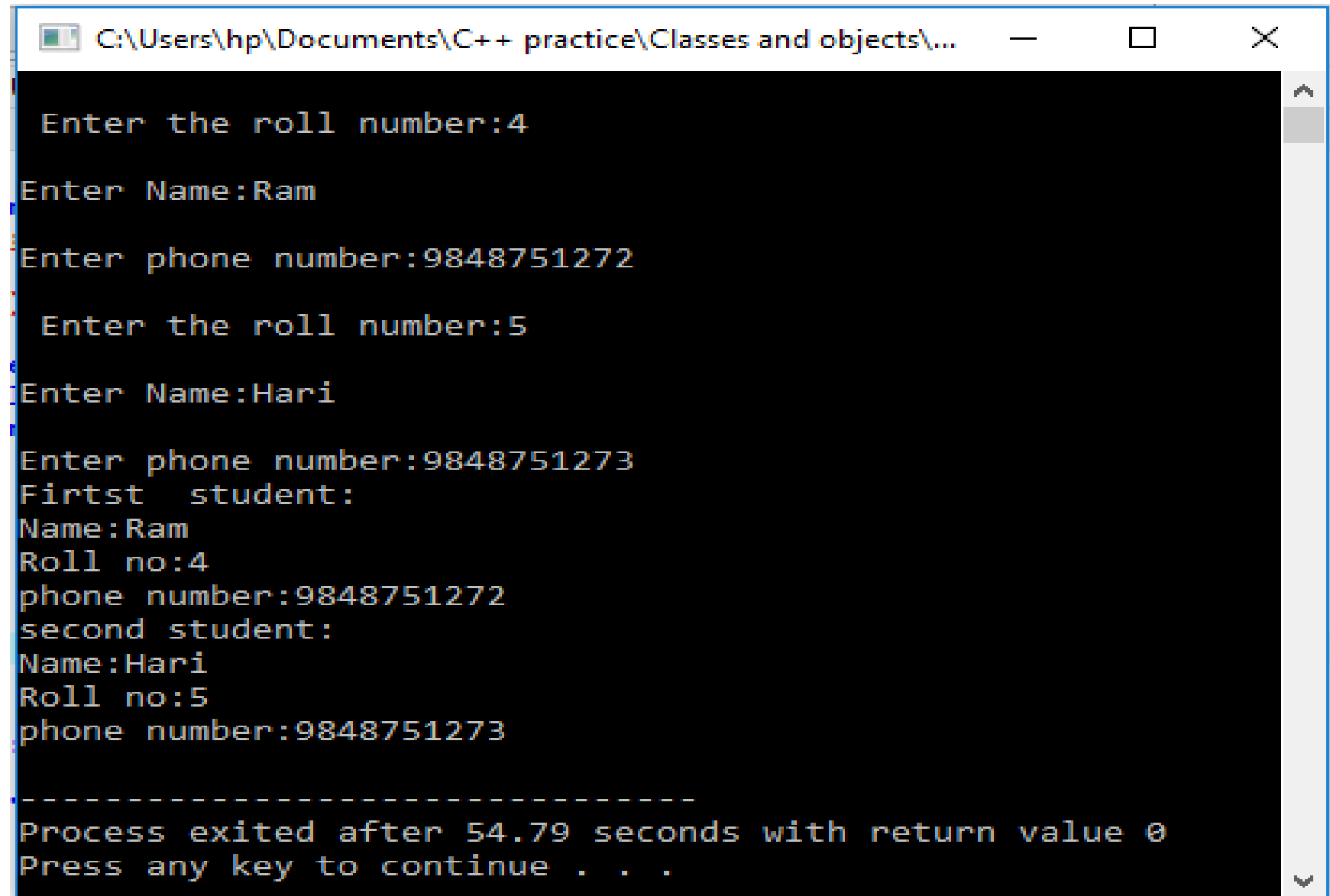
- **Practice work:** Write a program designing a class to represent student.
- Include the following members
 - Data members
 - Roll
 - Name
 - phone
 - Member functions
 - Getdata
 - showdata
- Define member function inside class
- Define member function outside class
- Make the functions inline also

Example:

```
//Defining function with in class definition
#include<iostream>
using namespace std;
class student
{
private:
    int roll;
    char name[20];
    char phone[10];
public:
    void getdata()
    {
        cout<<"\n Enter the roll number:";
        cin>>roll;
        cout<<"\nEnter Name:";
        cin>>name;
        cout<<"\nEnter phone number:";
        cin>>phone;
    }
    void showdata()
    {
        cout<<"Name:"<<name<<endl;
        cout<<"Roll no:"<<roll<<endl;
        cout<<"phone number:"<<phone<< endl;
    }
};

int main()
{
    student s1, s2;
    s1.getdata();
    s2.getdata();
    cout<<"Firtst student:"<<endl;
    s1.showdata();
    cout<<"second student:"<<endl;
    s2.showdata();
    return 0;
}
```

Output:



```
C:\Users\hp\Documents\C++ practice\Classes and objects\...  
Enter the roll number:4  
Enter Name:Ram  
Enter phone number:9848751272  
Enter the roll number:5  
Enter Name:Hari  
Enter phone number:9848751273  
Firtst student:  
Name:Ram  
Roll no:4  
phone number:9848751272  
second student:  
Name:Hari  
Roll no:5  
phone number:9848751273  
-----  
Process exited after 54.79 seconds with return value 0  
Press any key to continue . . .
```

Example:

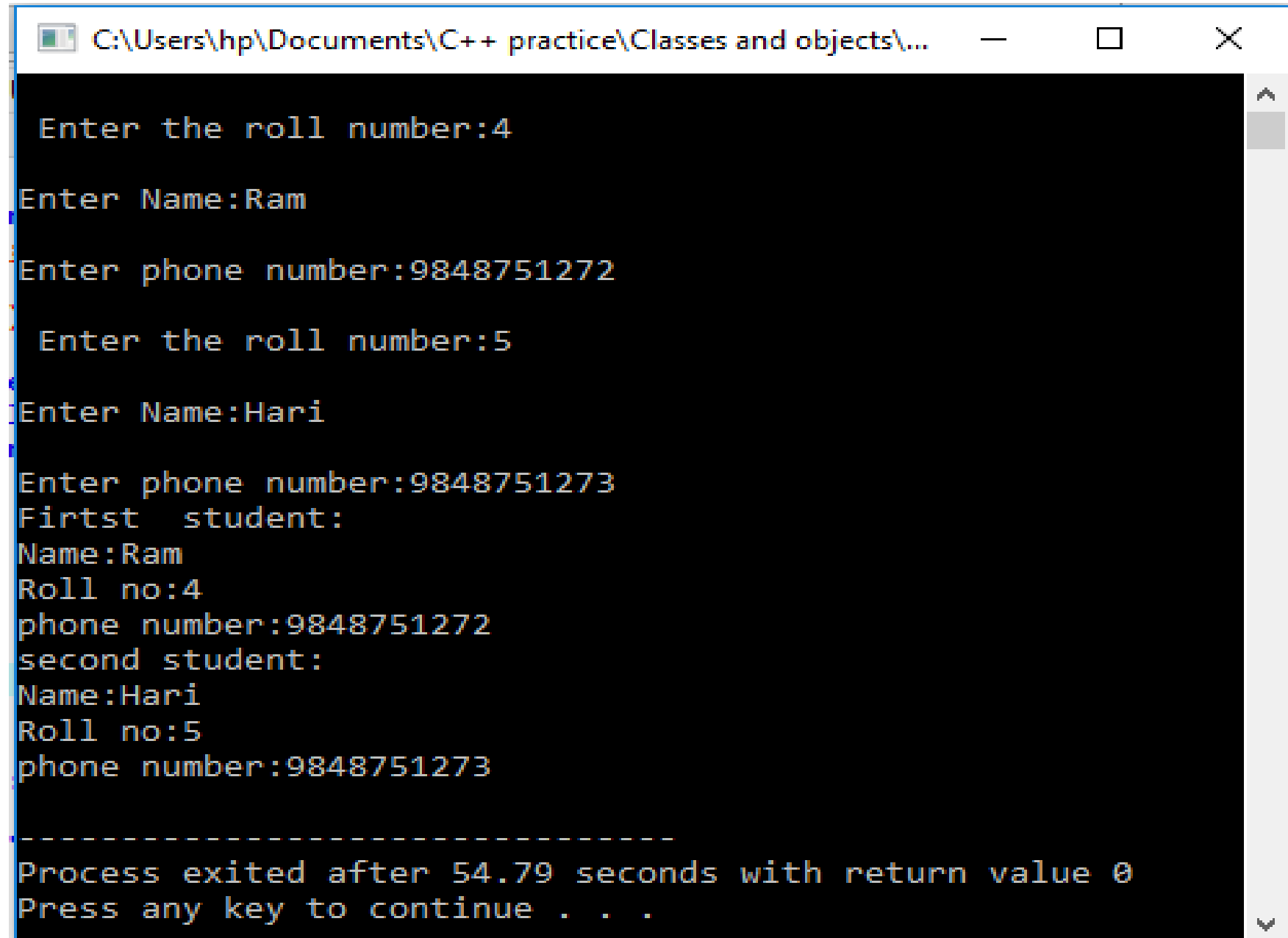
```
//Defining function outside in class definition
#include<iostream>
using namespace std;
class student
{
private:
    int roll;
    char name[20];
    char phone[10];
public:
    void getdata();
    void showdata();
};

void student:: getdata()
{
    cout<<"\n Enter the roll number:";
    cin>>roll;
    cout<<"\nEnter Name:";
    cin>>name;
    cout<<"\nEnter phone number:";
    cin>>phone;
}

void student:: showdata()
{
    cout<<"Name:"<<name<<endl;
    cout<<"Roll no:"<<roll<<endl;
    cout<<"phone number:"<<phone<< endl;
}

int main()
{
    student s1, s2;
    s1.getdata();
    s2.getdata();
    cout<<"Firtst student:"<<endl;
    s1.showdata();
    cout<<"second student:"<<endl;
    s2.showdata();
    return 0;
}
```

Output:



```
C:\Users\hp\Documents\C++ practice\Classes and objects\...

Enter the roll number:4
Enter Name:Ram
Enter phone number:9848751272

Enter the roll number:5
Enter Name:Hari
Enter phone number:9848751273
First student:
Name:Ram
Roll no:4
phone number:9848751272
second student:
Name:Hari
Roll no:5
phone number:9848751273

-----
Process exited after 54.79 seconds with return value 0
Press any key to continue . . .
```

CONSTRUCTORS AND DESTRUCTORS

- When an object is created all the members of the object are allocated memory spaces.
- Each objects has its individual copy of member variables.
- However the data members(member variables) are not initialized automatically.

- For initialization of data members we need to make member functions like `init()` as follows:

```
class test
{
    private:
        int data;
    public:
        void init(int d)
        {
            data =d;
        }
        //.....
};
//After creating the object the initialization is done as:
test t;
t.inti(5);
```

- **Problem:** If the programmer forgets to initialize the variable by calling the function `init()`, the program will produce unusual result.
- **Solution:** Constructor!

- What is constructor?
 - A constructor is a special member function that is executed automatically whenever an object is created.
 - It is used for automatic initialization.
 - Automatic initialization is the process of initializing object's data members when it is first created, without making a separate call to a member function.

- Some characteristics of constructors are:
 - The name of the constructor is same as the class name.
 - Constructors should be defined or declared in the public section
 - They do not have return types.
 - Like functions they can have default arguments.

- **Syntax:**

```
class class_name
{
    private:
        .....
        .....
    public:
        class_name();    //Constructor
        {
            .....
            .....
        }
}
```

- **Example:**

```
class rectangle
{
    private:
        int length;
        int breadth;
    public:
        rectangle() |
        {
            length =0;
            breadth=0;
        }
        .....
        .....
};
```

TYPES OF CONSTRUCTOR

- Default constructor
- Parameterized constructor
- Copy constructor
- Default copy constructor

- **Default Constructor:**

- A constructor that takes no arguments is called default constructor.

- **E.g.,:**

```
class rectangle
{
    private:
        int length;
        int breadth;
    public:
        rectangle() // Default constructor
        {
            length =0;
            breadth=0;
        }
        .....
        .....
};
```

- A default constructor is called automatically at the time of object creation (if no arguments are supplied) and does nothing more than initializing the data variables of the object to valid initial values.
- **Syntax:** `class_name object_name; // default constructor is invoked`
- **E.g.,:** `rectangle r1; // default constructor rectangle of the class rectangle is invoked.`
- **Note:** Compiler generates the default constructor if none of the constructors are defined.

- Parameterized constructors:

- The constructor that can take arguments are called parameterized constructors.

- E.g.,:

```
class rectangle
{
    private:
        int length;
        int breadth;
    public:
        rectangle(int l, int b) // parameterized constructor
        {
            length =l;
            breadth=b;
            .....
            .....
        };
};
```

- Parameterized constructors are used if it is necessary to initialize the various data elements of different objects with different values when they are created.
- This can be achieved by passing arguments to the constructor when the objects are created.
- Two ways:
 - By calling the constructor explicitly: e.g., `rectangle r1 = rectangle(5,6)`
 - By calling the constructor implicitly: e.g., `rectangle r1(5, 6);`

- **Copy constructors:**

- Copy constructor allows an object to be initialized with another object of the same class.
- It implies that the values stored in data members of an existing object can be copied into the data members of the object being constructed, provided the objects belong to the same class.
- A copy constructor has a single parameter of reference type that refers to the class itself as shown below:

- E.g.,:

```
//Defining copy constructor  
rectangle(rectangle& r)  
{  
    length = r.length;  
    breadth = r.breadth;  
}
```

- Once a copy constructor is defined, we can use copy constructor as :
 - `rectangle r2(r1) ;`
- It creates new object r2 and perform member-by-member copy of r1 into r2 .

- **Default copy constructor:**

- If you don't define copy constructor, the compiler creates a default copy constructor for each class which does a member-wise copy between objects.
- But it will do shallow copy, if class contains pointers then default copy constructor can not copy value pointed by pointer variable of one object into pointer variable of another object rather both of them will point to the same place.
- To handle this situation we need to create copy constructor.
- Default copy constructor can be invoked as below:

`rectangle r2 = r1;`

COMPILED BY: DABBAL S. MAHARA It creates new object r2 and perform member-by-member copy of r1 into r2.

- **Example:**

```
//program to illustrate the concept of constructor
#include<iostream>
using namespace std;
class rectangle
{
    private:
        int length;
        int breadth;
    public:
        rectangle()//default constructor
        {
            length = 0;
            breadth = 0;
        }
        rectangle(int l, int b)//parameterized constructor
        {
            length = l;
            breadth = b;
        }
        //Defining copy constructor
        rectangle(rectangle& r)
        {
            length = r.length;
            breadth = r.breadth;
        }
        int display()
        {
            cout<< "length:"<<length<<endl;
            cout<<"breadth:"<<breadth<<endl<<endl;
            return 0;
        }
};
```

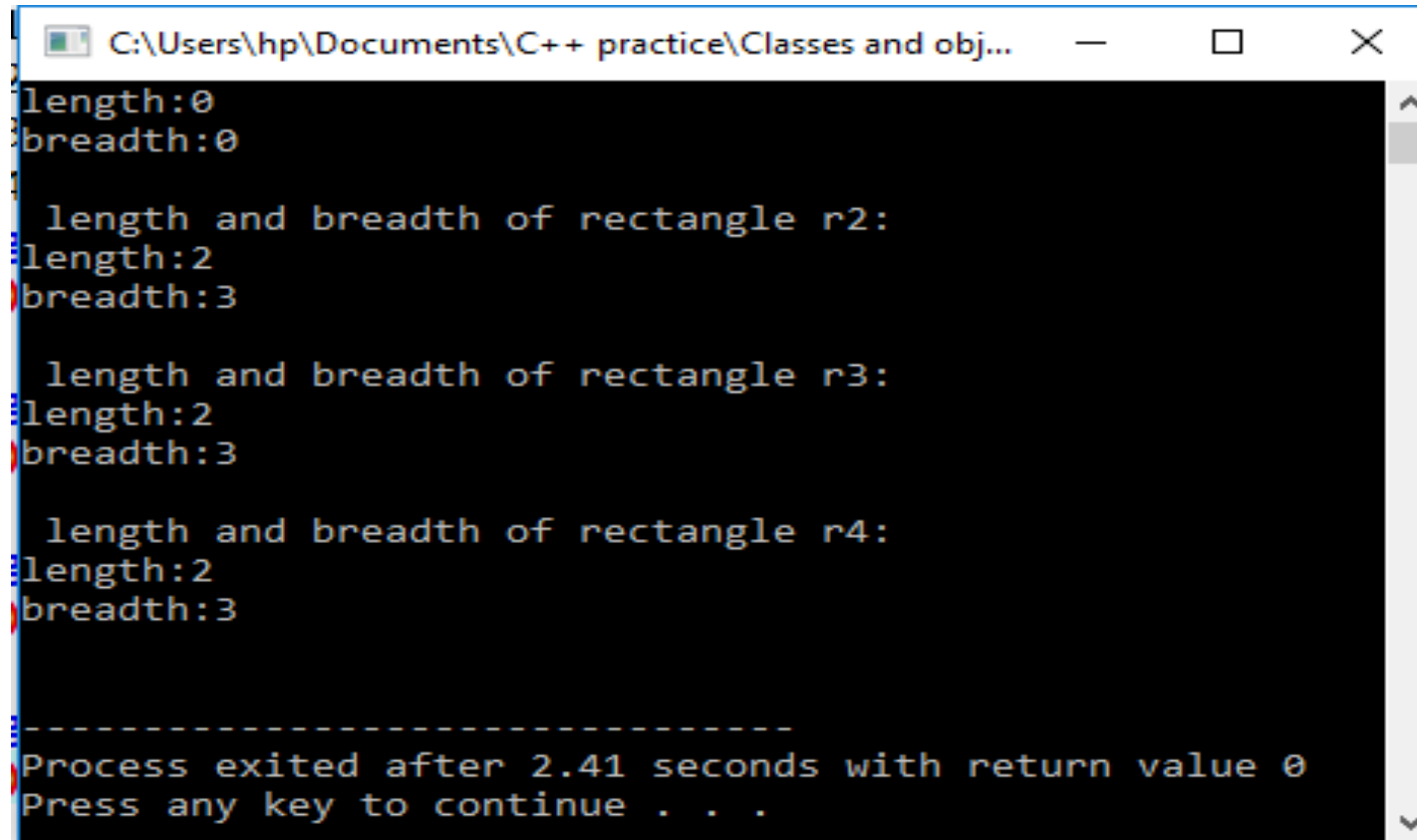
```
int main()
{
    rectangle r1;
    rectangle r2 (2, 3);
    rectangle r3(r2);    //r4 is copy of r2
    rectangle r4= r2;
    cout<< "length and breadth of rectangle r1:"<<endl;
    r1.display();

    cout<<" length and breadth of rectangle r2:"<<endl;
    r2.display();

    cout<<" length and breadth of rectangle r3:"<<endl;
    r3.display();

    cout<<" length and breadth of rectangle r4:"<<endl;
    r4.display();
}
```

- **Output:**



A screenshot of a Windows console window titled "C:\Users\hp\Documents\C++ practice\Classes and obj...". The console has a black background with white text. The output shows the initialization of a rectangle object with length 0 and breadth 0. Then, three more rectangles (r2, r3, r4) are created, each with length 2 and breadth 3. The output is formatted with indentation and line numbers on the left. At the bottom, a dashed line separates the program output from the system message "Process exited after 2.41 seconds with return value 0" and the prompt "Press any key to continue . . .".

```
length:0  
breadth:0  
  
    length and breadth of rectangle r2:  
length:2  
breadth:3  
  
    length and breadth of rectangle r3:  
length:2  
breadth:3  
  
    length and breadth of rectangle r4:  
length:2  
breadth:3  
  
-----  
Process exited after 2.41 seconds with return value 0  
Press any key to continue . . .
```

- **Constructor overloading:**
 - A class can have multiple constructors.
 - If more than one constructor is used in a class, it is known as constructor overloading.
 - Constructors can be overloaded either by:
 - different number of argument or
 - with different type of argument

- **Example:**

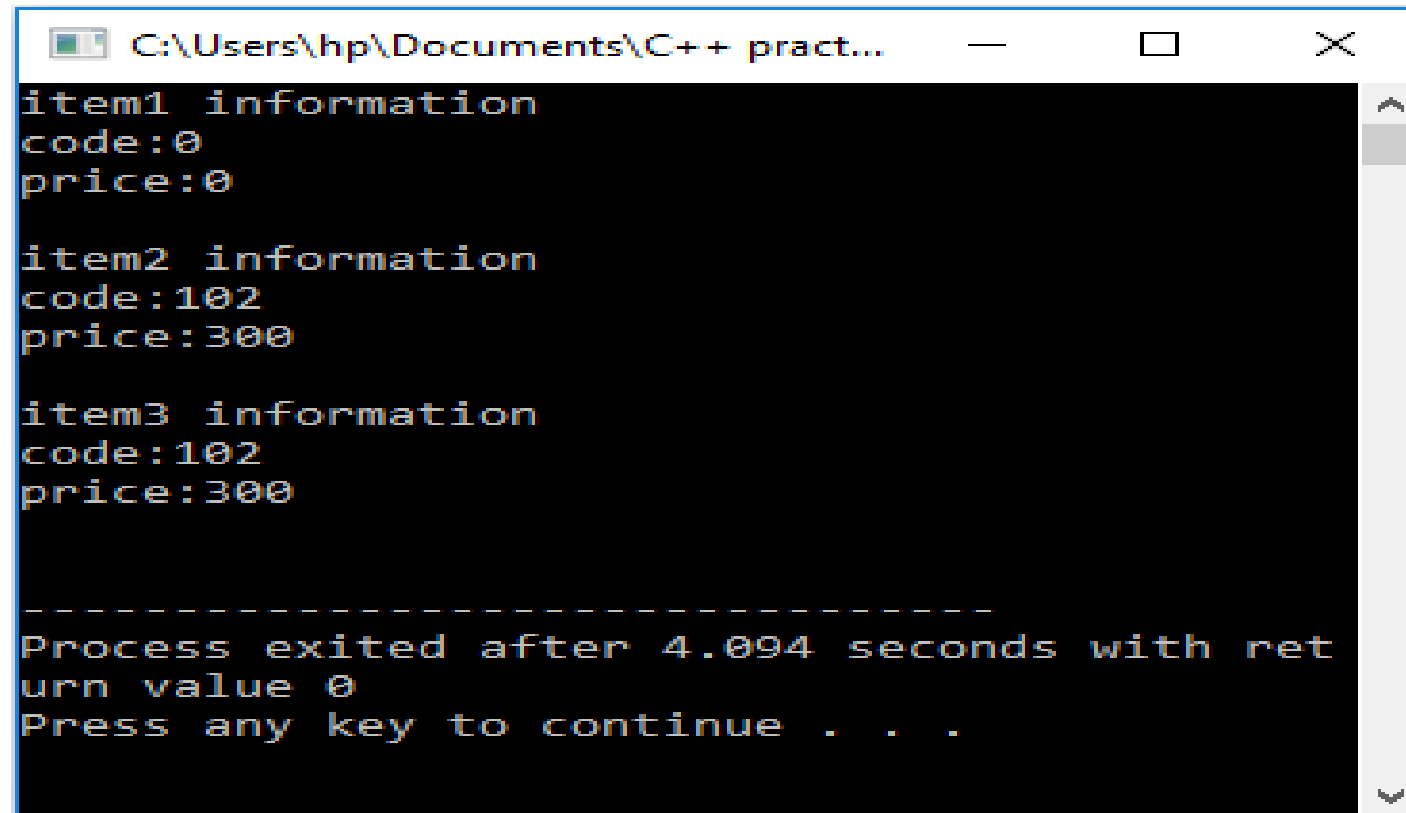
```
// program to illustrate the concept of constructor overloading
# include<iostream>
using namespace std;
class item
{
    int code, price;
public:
    item() //default constructor
    {
        code =0;
        price =0;
    }
    item(int c, int p) //parameterized constructor
    {
        code = c;
        price =p;
    }
    item(item &i)// copy constructor
    {
        code= i.code;
        price =i.price;
    }
    void display()
    {
        cout<<"code:"<<code<<endl<<"price:"<<price<<endl<<endl;
    }
};
```

```
int main()
{
    item i1;
    item i2(102,300);
    item i3(i2);
    cout<<"item1 information"<<endl;
    i1.display();

    cout<<"item2 information"<<endl;
    i2.display();

    cout<<"item3 information"<<endl;
    i3.display();
}
```


- **Output:**

A screenshot of a Windows console window titled "C:\Users\hp\Documents\C++ pract...". The window has a black background with yellow text. The output shows three items of information, each with a code and price. The first item has a code of 0 and a price of 0. The second and third items have a code of 102 and a price of 300. After the third item, there is a dashed line separator, followed by a message indicating the process exited after 4.094 seconds with a return value of 0, and a prompt to press any key to continue.

```
C:\Users\hp\Documents\C++ pract...  
item1 information  
code:0  
price:0  
  
item2 information  
code:102  
price:300  
  
item3 information  
code:102  
price:300  
  
-----  
Process exited after 4.094 seconds with ret  
urn value 0  
Press any key to continue . . .
```

DESTRUCTORS

- A destructor is a special member function that is executed automatically just before lifetime of an object is finished.
- The most common use of destructor is to destroy the memory that was allocated for the object by the constructor.
- In many cases program do not need user supplied destructors. In that case compiler supplies a default destructor.
- However, if program creates dynamic objects, then destructors should be defined explicitly in the program by the programmer

- **Destructors have the following characteristics:**
 - A Destructor has the same name as the constructor(which is the same as the class name) but is preceded by a tilde(~).
 - Like constructors, destructors do not have a return type.
 - Destructors takes no arguments. Hence we can use only one destructor in a class(i.e., we can not overload destructors).
 - Objects are destroyed in the reverse order of their creation.

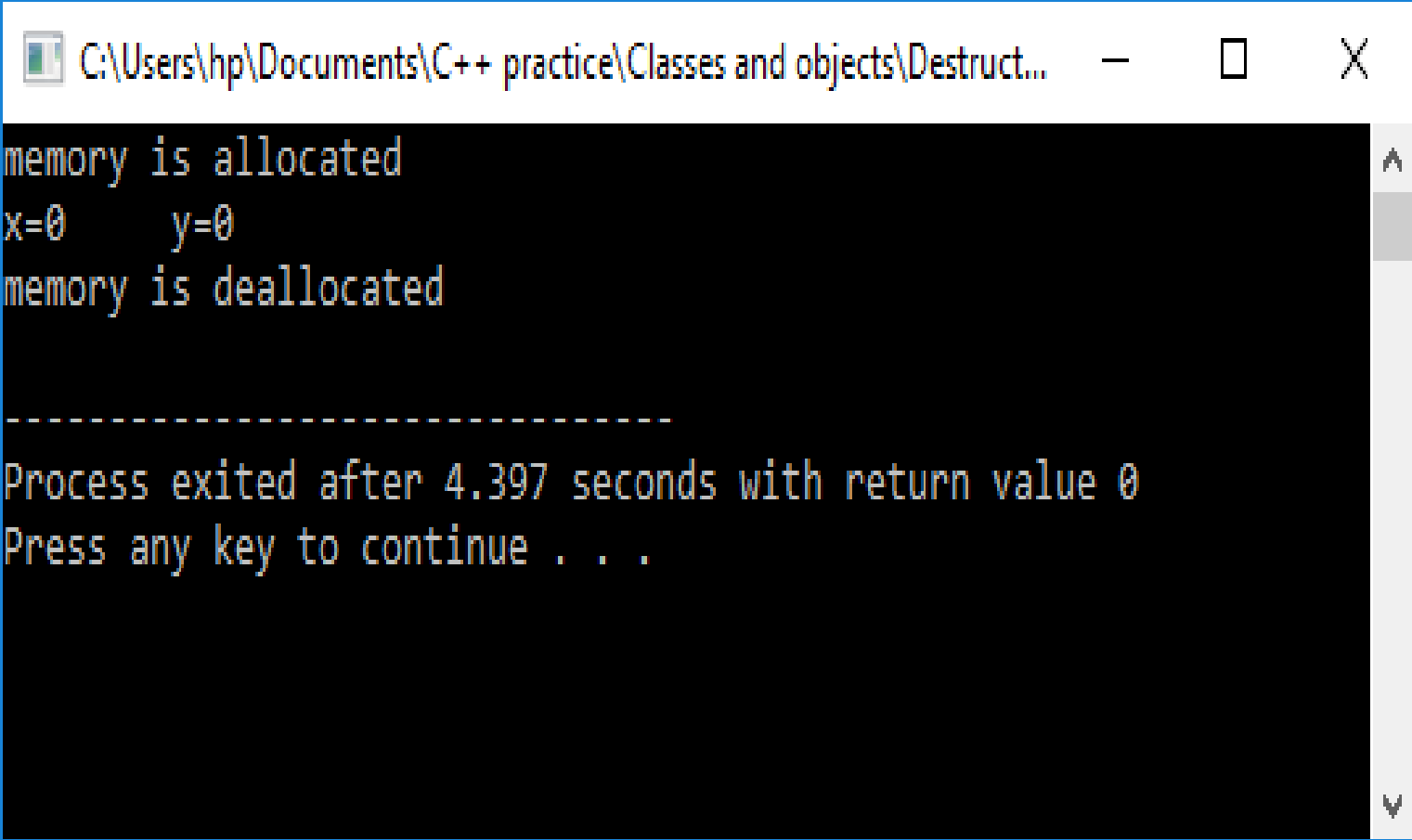
- **Syntax:**

```
class Test
{
    private:
        //...
    public:
        Test()
        {
            // body of constructor
        }
        ~Test()
        {
            //body of destructor
        }
};
```

- **Example:**

```
// program to illustrate the concept of destructor
#include<iostream>
using namespace std;
class Test
{
    private:
        int x, y;
    public:
        Test()
        {
            x=0;
            y=0;
            cout<<"memory is allocated"<<endl;
        }
        ~Test()
        {
            cout<<"x="<<x<<"\t"<<"y="<<y<<endl;
            cout<<"memory is deallocated"<<endl;
        }
};
int main()
{
    {
        Test p;
    } // life time of p is finished here, and destructor is called
    |
    return 0;
}
```

- **Output:**

A screenshot of a Windows command prompt window. The title bar shows the file path: C:\Users\hp\Documents\C++ practice\Classes and objects\Destruct... The window has standard minimize, maximize, and close buttons. The command prompt area has a black background with yellow text. The output is as follows:

```
memory is allocated  
x=0      y=0  
memory is deallocated  
  
-----  
Process exited after 4.397 seconds with return value 0  
Press any key to continue . . .
```

STATIC DATA MEMBERS

- Static data members declared inside class by using **static** keyword and must be defined outside the class using scope resolution operator.

- **Syntax:**

```
class A
{
    static int i; // static data member declaration
    public:
        //.....
};
int A::i=1; // definition of static data member i
```

- Only one copy of static member is created for the entire class and is shared by all the objects of that class.
- Used to maintain values common to entire class .

- **Example:**

```
//program for counting number of objects
#include<iostream>
using namespace std;
class student
{
    private:
        int roll;
        char name[20];
        static int count; //static data member
    public:
        void getdata()
        {
            cout<<"Enter roll number and name"<<endl;
            cin>>roll>>name;
            count++;
        }
        void displayCout()
        {
            cout<<"count:"<<count<<endl;
        }
};
```

```
//assignment of values to static data members
int student::count =0 ;
int main()
{
    student s1, s2, s3;
    //count will be 0
    s1.displayCout();
    s2.displayCout();
    s3.displayCout();
    //getting data for each object
    s1.getdata();
    s1.displayCout();

    s2.getdata();
    s2.displayCout();

    s3.getdata();
    s3.displayCout();
    return 0;
}
```


- **Output:**

```
count:0
count:0
count:0
Enter roll number and name
1 Ram
count:1
Enter roll number and name
2 Hari
count:2
Enter roll number and name
3 Sita
count:3
```

STATIC MEMBER FUNCTIONS

- Static member function can be defined by **using static keyword** and called by using scope resolution operator.
- Syntax:

```
class x
{
    public:
        static void f()
        {
            //body of static member function
        }
};
int main()
{
    x::f();// calling static member funtion
}
```

- **Features:**

- A static member function can be called even if no objects of the class exist.
- The **static** functions are accessed using the class name and the scope resolution operator ::.
- A static member function can only access static data member, and other static member functions.
- A static member function cannot access ordinary data members and member functions

- **Example:**

```
//program for counting number of objects
//static member function for displayinn count
#include<iostream>
using namespace std;
class student
{
    private:
        int roll;
        char name[20];
        static int count; //static data member
    public:
        void getdata()
        {
            cout<<"Enter roll number and name"<<endl;
            cin>>roll>>name;
            count++;
        }
        static void displayCout()
        {
            cout<<"count:"<<count<<endl;
        }
};
```

```
//assignment of values to static data members
int student::count =0 ;
int main()
{
    student s1, s2, s3;
    //count will be 0
    student::displayCout();

    //getting data for each object
    s1.getdata();
    student::displayCout();

    s2.getdata();
    student::displayCout();

    s3.getdata();
    student::displayCout();
    return 0;
}
```

- **Output:**

```
count:0
Enter roll number and name
1 Ram
count:1
Enter roll number and name
2 Hari
count:2
Enter roll number and name
3 Sita
count:3
-----
```

OBJECTS AS A FUNCTION ARGUMENTS

- Like any other variables, an object may be used as a function argument in three ways:
 - Pass-by-value
 - Pass-by-reference, and
 - Pass-by-pointer

- **Pass by value:**

- In pass by value a copy of the object is passed to the function. so, the changes made to the object inside the function do not affect the actual object.
- since all values of objects need to be copied into arguments of method invoked, it makes program slower when larger objects are used.
- An object can be passed by value to the function as below:

- Example:

```
// objects as function arguments(pass-by-value)
#include <iostream>
using namespace std;
class Demo
{
    private:
        int a;

    public:
        void set(int x)
        {
            a = x;
        }

        void sum(Demo ob1, Demo ob2)
        {
            a = ob1.a + ob2.a;
        }

        void print()
        {
            cout<<"Value of A : "<<a<<endl;
        }
};
```

```
int main()
{
    //object declarations
    Demo d1;
    Demo d2;
    Demo d3;

    //assigning values to the data member of objects
    d1.set(10);
    d2.set(20);

    //passing object d1 and d2
    d3.sum(d1,d2);

    //printing the values
    d1.print();
    d2.print();
    d3.print();

    return 0;
}
```

COMPILED BY: DABE

```
Value of A : 10
Value of A : 20
Value of A : 30
-----
```

- **Pass-by-reference:**

- In pass by reference, an address of the object is passed to the function, it makes program faster when using larger objects.
- The function works directly on the actual object used in the function call.
- So any changes made to object inside the function will reflect in the actual object.

- **Example:**

```
// objects as function arguments(pass by reference)
#include <iostream>
using namespace std;
class Demo
{
    private:
        int a;

    public:
        void set(int x)
        {
            a = x;
        }

        void sum(Demo &ob1, Demo &ob2)
        {
            a = ob1.a + ob2.a;
        }

        void print()
        {
            cout<<"Value of A : "<<a<<endl;
        }
};
```

```
int main()  
{  
    //object declarations  
    Demo d1;  
    Demo d2;  
    Demo d3;  
  
    //assigning values to the data member of objects  
    d1.set(10);  
    d2.set(20);  
  
    //passing object d1 and d2  
    d3.sum(d1,d2);  
  
    //printing the values  
    d1.print();  
    d2.print();  
    d3.print();  
  
    return 0;  
}
```

```
Value of A : 10  
Value of A : 20  
Value of A : 30
```

- **Pass-by-pointer:**

- Also works directly on the actual object used in the function call.

```
//objects as function arguments(pass-by-pointer)
#include <iostream>
using namespace std;
class Demo
{
private:
    int a;

public:
    void set(int x)
    {
        a = x;
    }

    void sum(Demo *ob1, Demo *ob2)
    {
        a = ob1->a + ob2->a;
    }

    void print()
    {
        cout<<"Value of A : "<<a<<endl;
    }
};
```

```
int main()  
{  
    //object declarations  
    Demo d1;  
    Demo d2;  
    Demo d3;  
  
    //assigning values to the data member of objects  
    d1.set(10);  
    d2.set(20);  
  
    //passing object d1 and d2  
    d3.sum(&d1,&d2);  
  
    //printing the values  
    d1.print();  
    d2.print();  
    d3.print();  
  
    return 0;  
}
```

```
Value of A : 10  
Value of A : 20  
Value of A : 30
```

RETURNING OBJECTS FROM THE FUNCTIONS

- A function not only receives objects as arguments but also can return them.
- A function can return objects also in three ways:
 - Return-by- value
 - Return-by- reference, and
 - Return-by-pointer.

- **Return-by-value:** A copy of the object is returned to the function call.

```
//Returning objects by value  
// program that adds two objects of complex class  
#include<iostream>  
using namespace std;  
class complex  
{  
    private:  
        int real, img;  
    public:  
        void getdata()  
        {  
            cout<<"enter the real and imaginary part of complex number"<<endl;  
            cin>>real>>img;  
        }  
        void display()  
        {  
            cout<<real<<"+"<<img<<endl;  
        }  
//addcomplex() method returns the object temp by value  
        complex addcomplex(complex c)  
        {  
            complex temp;  
            temp.real= real + c.real;  
            temp.img=img+c.img;  
            return temp;  
        }  
};
```

```

int main()
{
    complex c1, c2, c3;

    c1.getdata();

    c2.getdata();

    c3= c2.addcomplex(c1);

    cout<<"Addition=";

    c3.display();

    return 0;
}

```

```

enter the real and imaginary part of complex number
12
8
enter the real and imaginary part of complex number
12
9
Addition=24+i17

```

- **Return-by-reference:** An address of the object is returned to the function call.

```
//returning objects by reference
// program that adds two objects of complex class
#include<iostream>
using namespace std;
class complex
{
    private:
        int real, img;
    public:
        void getdata()
        {
            cout<<"enter the real and imaginary part of complex number"<<endl;
            cin>>real>>img;
        }
        void display()
        {
            cout<<real<<" + i" <<img<<endl;
        }
        ///addcomplex() method returns the object c by reference
        complex addcomplex(complex &c)
        {
            c.real= real + c.real;
            c.img=img+c.img;
            return c;
        }
};
```

```

int main()
{
    complex c1, c2, c3;

    c1.getdata();

    c2.getdata();

    c3= c2.addcomplex(c1);

    cout<<"Addition=";

    c3.display();

    return 0;
}

```

```

enter the real and imaginary part of complex number
2
6
enter the real and imaginary part of complex number
4
3
Addition=6+i9

```

- **Return-by-pointer:** It also returns address of the object to the function call.

```
//Returning objects by pointer
// program that adds two objects of complex class
#include<iostream>
using namespace std;
class complex
{
    private:
        int real, img;
    public:
        void getdata()
        {
            cout<<"enter the real and imaginary part of complex number"<<endl;
            cin>>real>>img;
        }
        void display()
        {
            cout<<real<<"+"<<img<<endl;
        }
        complex *addcomplex(complex *c)
        {
            c->real= real + c->real;
            c->img=img+c->img;
            return c;
        }
};
```

```
int main()
{
    complex c1, c2, *c3;

    c1.getdata();

    c2.getdata();

    c3= c2.addcomplex(&c1);

    cout<<"Addition=";

    c3->display();

    return 0;
}
```

```
enter the real and imaginary part of complex number
4
5
enter the real and imaginary part of complex number
6
7
Addition=10+i12
```

Home Work

79

1. Write a program to read and display 10 objects of item class containing data member item, name ,code and price.
2. Write a program designing a class to represent a bank account. Include the following members
 - Data members: name of depositor, account number, type of account
 - Member functions: To assign initial values, To display name and account type
3. Construct a class named intamount with the following members to calculate interest and amount separately on the basis of given principle , rate and time.
 - Data members: principle, time, rate
 - Function members: getdata, interest, amount.
4. Define a class circle with the following data members and member functions:
 - Data members: radius and area
 - Member functions: getdata, calcArea and display.

THANK YOU !